

PHP — Part 2

Complete Practical PHP for Your Semester Project

4-Hour Session · Web Techniques

Prof. Soufiane Hourri

A Recap

B MySQL

C Files

D Sessions

E Cookies

F Security

Today's 4-Hour Plan

8 parts · From MySQL to a complete working project

A

Quick Recap

Reconnect with forms, GET/POST, validation, DB idea

B

MySQL CRUD

INSERT, SELECT, UPDATE, DELETE — full CRUD in PHP

C

File Structure

Project files, include/require, page flow

D

Sessions + Login

Login logic, page protection, logout

E

Cookies

setcookie(), \$_COOKIE, Sessions vs Cookies

F

Security & Debug

password_hash, safe habits, var_dump, die()

G

Mini Project

End-to-end workflow — how everything connects

H

Recap + Homework

Summary, next steps, mini-project brief

A

Part A

Quick Recap

Reconnect with what you already know before we go further

What We Covered in Session 1

You already know these — let's confirm before we continue

PHP Fundamentals

- PHP syntax: `<?php ... ?>` tags, echo, comments
- Variables, data types, operators
- Conditions: if / elseif / switch
- Loops: for, while, foreach
- Arrays: indexed and associative

Forms + Database Intro

- HTML forms: action + method
- `$_POST` and `$_GET` to receive data
- Basic validation with `isset()` / `strlen()`
- DB connection with `mysqli_connect()`
- First SELECT and INSERT queries

Forms Recap — GET vs POST

Two ways to send data from a form to PHP

```
<!-- GET: data visible in URL -->
<form action="search.php" method="GET">
  <input type="text" name="keyword">
  <button>Search</button>
</form>
<!-- URL becomes: search.php?keyword=hello -->
```

```
// In search.php:
$kw = $_GET["keyword"]; // "hello"
```

```
<!-- POST: data hidden, for sensitive/large data -->
<form action="save.php" method="POST">
  <input type="text" name="name">
  <input type="password" name="password">
  <button>Submit</button>
</form>
```

```
// In save.php:
$name = $_POST["name"];
$pass = $_POST["password"];
```

GET vs POST — Remember

- 1 GET puts data in the URL bar — anyone can see it. Use for searches.
- 2 POST hides data — use for login forms, registration, saving records.
- 3 `$_GET['key']` reads URL parameters like `?id=5` or `?name=ahmed`.
- 4 `$_POST['key']` reads hidden form data sent via `method='POST'`.
- 5 Always check if a key exists before reading it with `isset()`.
- 6 The form action tells PHP which file receives the data.

The Big Picture — How Your Project Works

Data flows from browser → PHP → database → back to browser



What this looks like in your project files

register.php

HTML form for new student/product/user

save.php

Receives POST, validates, runs INSERT into DB

list.php

Runs SELECT, loops and displays results in a table

edit.php

Loads record by ID (SELECT WHERE id=?), shows edit form

update.php

Receives POST from edit form, runs UPDATE query

delete.php

Receives GET id, runs DELETE query, redirects

B

Part B

MySQL + PHP

INSERT, SELECT, UPDATE, DELETE — complete CRUD with real examples

Step 1 — Connect to the Database

Always the first step. Put this in db_connect.php and reuse it everywhere.

```
<?php
// db_connect.php – create this file ONCE, reuse everywhere

$host  = "localhost"; // almost always localhost
$user  = "root";       // your MySQL username
$pass  = "";           // your MySQL password (often empty in
                        // XAMPP)
$dbname = "my_project"; // the name of your database in
                        // phpMyAdmin

$conn = mysqli_connect($host, $user, $pass, $dbname);

// Always check the connection worked
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
// If we reach here – connection is OK
?>
```

DB Connection — Key Points

- 1 `mysqli_connect()` takes 4 arguments: host, username, password, database name.
- 2 On XAMPP: host='localhost', user='root', pass='' (empty) is the default.
- 3 Always check the connection with `if(!$conn)` — never assume it worked.
- 4 `die()` stops the script and shows the error message. Use only during dev.
- 5 Save this as `db_connect.php` and use `include/require` in every other file.
- 6 Never copy-paste the connection code — one file, reused everywhere.

INSERT — Save Form Data to the Database

Two files: the HTML form and the PHP that processes it

```
<!-- register.php — the form -->
<form action="save.php" method="POST">
  <input type="text" name="name" placeholder="Name">
  <input type="text" name="email" placeholder="Email">
  <button type="submit">Save</button>
</form>
```

```
// save.php — receives and saves the data
<?php
require_once "db_connect.php";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST["name"]);
    $email = trim($_POST["email"]);

    if (empty($name) || empty($email)) {
        die("All fields are required.");
    }

    $sql = "INSERT INTO students (name, email)
        VALUES ('$name', '$email')";

    if (mysqli_query($conn, $sql)) {
        header("Location: list.php");
        exit;
    } else {
        echo "Error: " . mysqli_error($conn);
    }
}
?>
```

INSERT — Step by Step

- 1 The form uses method='POST' so data is hidden. action='save.php' tells it where to send.
- 2 `$_SERVER['REQUEST_METHOD'] === 'POST'` confirms the form was actually submitted.
- 3 `trim()` removes accidental spaces at start/end of input.
- 4 `empty()` checks if a field is blank — always validate before saving.
- 5 `mysqli_query($conn, $sql)` runs the INSERT and returns true/false.
- 6 `header('Location: list.php')` redirects after saving — always add `exit` after it.

SELECT — Fetch and Display Data

Read all records from the database and show them in an HTML table

```
<?php
// list.php — show all students
require_once "db_connect.php";

$sql      = "SELECT * FROM students ORDER BY id DESC";
$result = mysqli_query($conn, $sql);
?>

<table border="1">
  <tr>
    <th>ID</th><th>Name</th><th>Email</th><th>Actions</th>
  </tr>
  <?php while ($row = mysqli_fetch_assoc($result)): ?>
    <tr>
      <td><?= htmlspecialchars($row["id"]) ?></td>
      <td><?= htmlspecialchars($row["name"]) ?></td>
      <td><?= htmlspecialchars($row["email"]) ?></td>
      <td>
        <a href="edit.php?id=<?= $row['id'] ?>">Edit</a>
        <a href="delete.php?id=<?= $row['id'] ?>">Delete</a>
      </td>
    </tr>
  </tr>
  <?php endwhile; ?>
</table>
```

SELECT + Display — Key Points

- 1 `mysqli_query()` runs the SELECT and returns a result object (not the data yet).
- 2 `mysqli_fetch_assoc()` reads ONE row as a named array — call it in a while loop.
- 3 The while loop runs once per row until there are no more rows.
- 4 `$row['name']` accesses a column by its name. Make sure the name matches the DB.
- 5 `htmlspecialchars()` converts `<` `>` `'` `"` to safe HTML — always use when displaying user data.
- 6 Edit and Delete links pass the row's ID in the URL: `edit.php?id=3`

SELECT with WHERE — Load One Record by ID

Used in edit pages: read the record to pre-fill the form

```
<?php
// edit.php — load one student record by ID
require_once "db_connect.php";

// 1. Get the id from the URL: edit.php?id=5
if (!isset($_GET["id"])) {
    die("No ID provided.");
}
$id = (int) $_GET["id"]; // (int) forces it to be a number

// 2. Fetch the record
$sql = "SELECT * FROM students WHERE id = $id";
$result = mysqli_query($conn, $sql);
$row = mysqli_fetch_assoc($result);

if (!$row) {
    die("Record not found.");
}
?>

<!-- 3. Show the form pre-filled with current values -->
<form action="update.php" method="POST">
    <input type="hidden" name="id" value="<?= $row['id'] ?>">
    <input type="text" name="name" value="<?=
htmlspecialchars($row['name']) ?>">
    <input type="text" name="email" value="<?=
htmlspecialchars($row['email']) ?>">
    <button>Update</button>
</form>
```

WHERE + Edit Form — Key Points

- 1 The ID comes from the URL: ?id=5. Always check with `isset()` before using it.
- 2 `(int)$_GET['id']` converts the URL value to an integer — prevents injection.
- 3 `SELECT * FROM table WHERE id = $id` fetches ONLY that one row.
- 4 If no row found, show an error with `die()` — don't continue with empty data.
- 5 Pre-fill the form with `value='<?= $row[...] ?>'` so the user sees the current value.
- 6 Use a hidden input for the ID so `update.php` knows which record to change.

UPDATE — Save Changes to an Existing Record

update.php receives the edited form and runs UPDATE SQL

```
<?php
// update.php — save the edited record
require_once "db_connect.php";

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    header("Location: list.php");
    exit;
}

$id    = (int)    $_POST["id"];
$name  = trim(    $_POST["name"]);
$email = trim(    $_POST["email"]);

if (empty($name) || empty($email)) {
    die("All fields are required.");
}

$sql = "UPDATE students
        SET name='$name', email='$email'
        WHERE id=$id";

if (mysqli_query($conn, $sql)) {
    header("Location: list.php");
    exit;
} else {
    echo "Error: " . mysqli_error($conn);
}
```

UPDATE — Critical Rules

- 1 Always check REQUEST_METHOD — if not POST, redirect to list.php.
- 2 Get the ID from \$_POST['id'] (the hidden field in the edit form).
- 3 Use SET col1='value', col2='value' to update multiple columns.
- 4 WHERE id=\$id is MANDATORY — without it you update ALL rows in the table!
- 5 After a successful UPDATE, always redirect — never leave the user on a processing page.
- 6 Validate inputs (empty, trim) before running the UPDATE, same as INSERT.

DELETE — Remove a Record

delete.php reads an ID from the URL, deletes the row, and redirects

```
<?php
// delete.php – delete a student by id
require_once "db_connect.php";

// Check that an id was provided
if (!isset($_GET["id"])) {
    header("Location: list.php");
    exit;
}

$id = (int) $_GET["id"];

// Run the DELETE
$sql = "DELETE FROM students WHERE id = $id";

if (mysqli_query($conn, $sql)) {
    // Success – go back to the list
    header("Location: list.php");
    exit;
} else {
    echo "Delete failed: " . mysqli_error($conn);
}
?>
```

DANGER — Never Forget WHERE

DELETE FROM students;
↳ Deletes ALL rows!

DELETE FROM students WHERE id=3;
↳ Deletes only row 3.

DELETE — Key Points

- 1 DELETE uses GET (from the list.php link) — no form needed.
- 2 (int)\$_GET['id'] ensures the ID is a safe integer.
- 3 WHERE id=\$id is critical — without it, ALL rows are deleted.
- 4 After delete, always redirect to list.php with header() + exit.
- 5 In real projects, add a confirm() dialog before triggering delete.
- 6 The delete file should only delete — never display a page itself.

Essential Functions — isset, empty, trim, htmlspecialchars

You will use these in every PHP file you write

```
<?php
// isset() — does this variable / key exist?
if (isset($_POST["name"])) {
    echo "Name was submitted";
}
// empty() — is it blank / zero / null?
if (empty($_POST["name"])) {
    echo "Name cannot be empty";
}
// trim() — remove spaces from start and end
$name = trim($_POST["name"]);
// " Ahmed " becomes "Ahmed"

// htmlspecialchars() — make output safe for HTML display
$safe = htmlspecialchars($name);
// <script> becomes &lt;script&gt; — harmless in HTML

// Typical pattern you will use constantly:
if (isset($_POST["submit"])) {
    $name = trim($_POST["name"]);
    if (empty($name)) {
        $error = "Name is required.";
    } else {
        $safe_name = htmlspecialchars($name);
        echo "Hello, $safe_name!";
    }
}
```

4 Functions You Must Know

- 1 isset() — checks the key exists and is not null. Use before reading any \$_POST or \$_GET key.
- 2 empty() — true if value is '', 0, null, false, or []. Use for required field validation.
- 3 trim() — removes invisible spaces. User might type ' Ahmed ' — trim gives 'Ahmed'.
- 4 htmlspecialchars() — converts < > & ' " to safe HTML codes. Always use when displaying user data.
- 5 These 4 together protect your project from blank inputs and basic attacks.
- 6 Order: isset() → trim() → empty() → htmlspecialchars() before display.

header() Redirect + exit — Control Page Flow

After saving, updating, or deleting — always redirect the user

```
<?php
// Redirect to another page
header("Location: list.php");
exit; // ALWAYS add exit after header()

// Redirect with a message in the URL
header("Location: list.php?msg=saved");
exit;

// In list.php — show the message
if (isset($_GET["msg"])) {
    echo "<p>Record saved successfully!</p>";
}

// IMPORTANT: header() must be called BEFORE
// any output (echo, HTML, spaces, blank lines)

// ❌ This causes "headers already sent" error:
echo "done";
header("Location: list.php"); // too late!

// ✅ Correct:
header("Location: list.php");
exit;
// No echo before header()
?>
```

header() — Rules

- 1 header('Location: page.php') sends a redirect instruction to the browser.
- 2 Always write exit; immediately after header() — it stops the script from continuing.
- 3 You can pass a message in the URL: ?msg=saved and read it with \$_GET['msg'].
- 4 CRITICAL: header() must come before any echo or HTML output, even a blank line.
- 5 If you see 'headers already sent' — you have echo/HTML before the header() call.
- 6 Use header() after every INSERT, UPDATE, and DELETE to avoid double-submissions.

Common CRUD Mistakes — Avoid These!

The most frequent errors beginners make with MySQL in PHP

X

Forgetting WHERE in UPDATE or DELETE

Always write WHERE id=\$id — otherwise ALL rows change!

X

Not checking isset() before \$_POST/\$_GET

Always: if(isset(\$_POST['name'])) before reading

X

echo before header() redirect

Never output anything before header('Location: ...')

X

Forgetting exit; after header()

Always: header('Location: x.php'); exit;

X

Displaying data without htmlspecialchars()

Always wrap: echo htmlspecialchars(\$row['name'])

X

Not validating empty fields before INSERT

Check empty() before running any SQL query

C

Part C

File Structure

How to organize your PHP project files — and why it matters

Typical PHP Project File Structure

Page (HTML + PHP)

index.php

Home page / welcome page

register.php

Registration form (HTML)

list.php

Shows all records in a table (SELECT)

update.php

Processes edit POST, runs UPDATE

login.php

Login form (HTML)

dashboard.php

Protected page — only for logged-in users

Logic (processing only)

db_connect.php

Database connection — included everywhere

save.php

Processes registration POST, runs INSERT

edit.php

Shows edit form pre-filled with one record

delete.php

Reads ?id from URL, runs DELETE, redirects

check_login.php

Validates credentials, sets session, redirects

logout.php

Destroys session, redirects to login.php

Utility (shared)

include, require, require_once — Reuse Files

Stop copying code — include it once and use it everywhere

```
<?php
// include — inserts another PHP file at this point
// If file missing: WARNING, script continues
include "header.php";

// require — inserts file; if missing: FATAL ERROR, stops
require "db_connect.php";

// require_once — like require, but includes only ONCE
// even if called multiple times (prevents duplicates)
require_once "db_connect.php";

// — Typical usage in every project file —————
<?php
require_once "db_connect.php"; // get $conn
require_once "includes/header.php"; // show nav/header HTML

// ... your page logic here ...

require_once "includes/footer.php"; // show footer HTML
?>

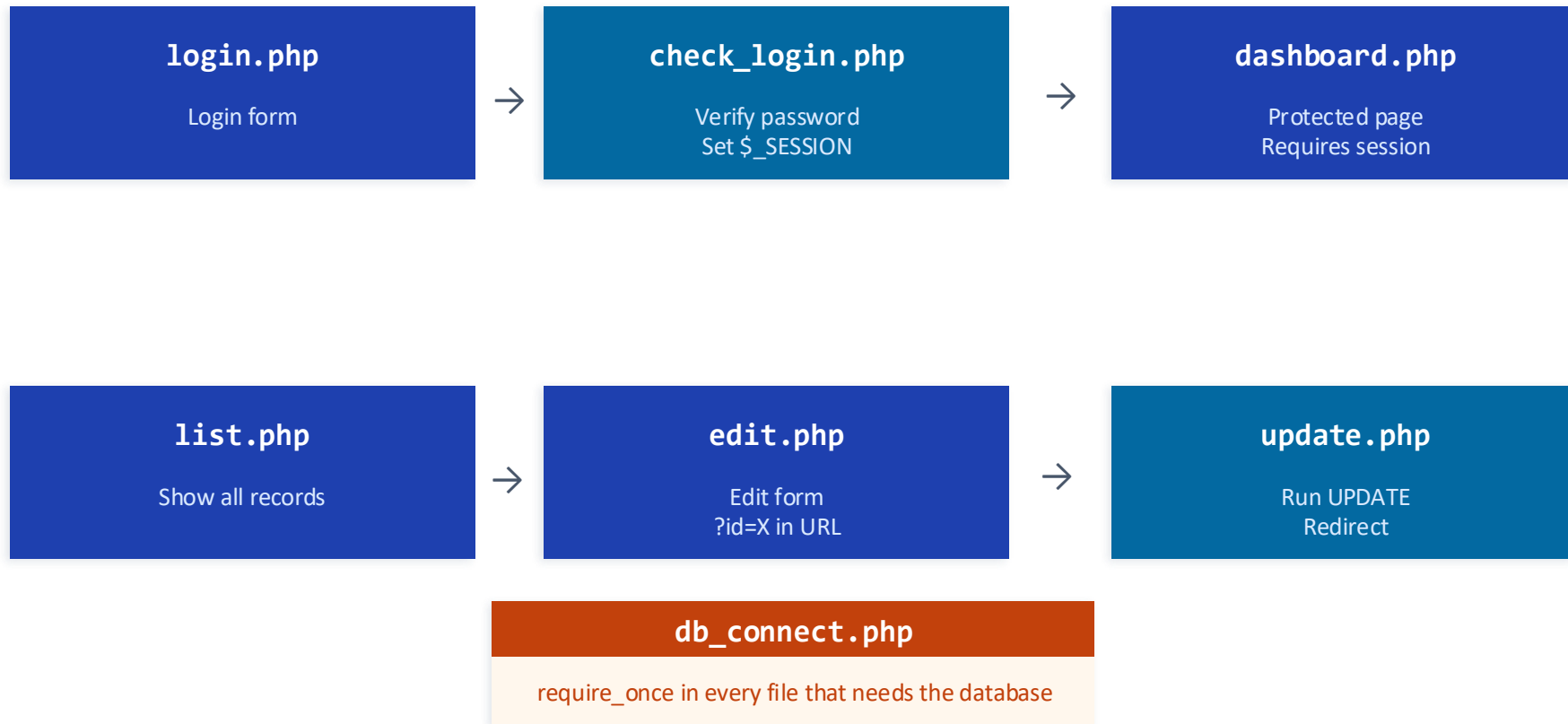
// — Why this matters in your project —————
// Without include/require:
//   copy-paste db connection code in ALL 12 files
//   change password? Must update ALL 12 files!
// With require_once "db_connect.php":
//   change ONCE → ALL files updated automatically
```

include vs require vs require_once

- 1 include — inserts the file; if not found, shows a warning but continues running.
- 2 require — inserts the file; if not found, shows fatal error and stops the script.
- 3 require_once — same as require, but guarantees the file is only included once.
- 4 Use require_once for db_connect.php to ensure \$conn is available everywhere.
- 5 Use require_once for shared HTML: header.php, footer.php, navbar.php.
- 6 Rule: always use require_once for anything critical (DB, auth checks).

How Pages Connect to Each Other

Visualise the flow before you write a single line of code



D

Part D

Sessions + Login

Login logic, protecting pages, and logout — the auth core of every project

What Is a Session?

A way to remember who the user is across multiple pages

```
<?php
// session_start() – MUST be the very first line
// before ANY output (HTML, echo, spaces)
session_start();

// Store data in the session
$_SESSION["user_id"]    = 5;
$_SESSION["user_name"]  = "Ahmed";
$_SESSION["role"]       = "admin";

// Read session data on ANY page
echo "Hello, " . $_SESSION["user_name"];

// Check if user is logged in
if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit;
}

// Remove one session variable
unset($_SESSION["user_name"]);

// Destroy the entire session (on logout)
session_destroy();
?>
```

Sessions — How They Work

- 1 A session stores data on the SERVER, not in the browser — it is secure.
- 2 PHP gives the user a unique session ID (stored as a cookie in the browser).
- 3 `session_start()` MUST be on line 1 — before any HTML or echo.
- 4 `$_SESSION` is a superglobal array — accessible on every page that calls `session_start()`.
- 5 Sessions expire when the browser closes (by default) — they are not permanent.
- 6 Use sessions for login state, user ID, role — things that must stay across pages.

Full Login Flow — Form → Check → Session → Protect

The complete login system for your semester project

```
<?php // check_login.php
session_start();
require_once "db_connect.php";

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    header("Location: login.php"); exit;
}

$email = trim($_POST["email"]);
$pass  = $_POST["password"];

// Fetch user by email
$sql    = "SELECT * FROM users WHERE email='$email'";
$result = mysqli_query($conn, $sql);
$user   = mysqli_fetch_assoc($result);

if ($user && password_verify($pass, $user["password"])) {
    // Correct credentials — create session
    $_SESSION["user_id"]   = $user["id"];
    $_SESSION["user_name"] = $user["name"];
    header("Location: dashboard.php");
    exit;
} else {
    // Wrong email or password
    header("Location: login.php?error=1");
    exit;
}
?>
```

Login Flow — Step by Step

- 1 Always check REQUEST_METHOD first — redirect if not POST.
- 2 Fetch the user from the DB by email — get the full row including hashed password.
- 3 password_verify() compares the typed password with the stored hash (never compare plain text!).
- 4 If correct: store user_id and user_name in \$_SESSION, redirect to dashboard.
- 5 If wrong: redirect back to login.php with ?error=1 so the form can show a message.
- 6 In login.php: check isset(\$_GET['error']) and display 'Invalid credentials'.

Protecting Pages + Logout

How to block access to restricted pages and log the user out

```
<?php
// dashboard.php – a protected page
session_start();

// The guard – put this at the TOP of every protected page
if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit;
}

// If we reach here – user is logged in
$username = $_SESSION["user_name"];
echo "<h1>Welcome, $username!</h1>";
?>

<?php
// logout.php – destroy the session and redirect
session_start();
session_unset();      // clear all session variables
session_destroy();     // destroy the session on the server
header("Location: login.php");
exit;
?>

<!-- In your navbar: -->
<a href="logout.php">Logout</a>
```

Protection + Logout — Rules

- 1 Copy the 4-line guard (session_start + isset check + redirect + exit) to EVERY protected page.
- 2 If the session variable is missing — user is not logged in — redirect immediately to login.
- 3 For logout: call session_start() first, then session_unset(), then session_destroy().
- 4 session_unset() clears the variables; session_destroy() removes the session file on server.
- 5 Always redirect after logout — never leave the user on logout.php.
- 6 Test: clear cookies in your browser and try accessing dashboard directly — should redirect.

Common Session Mistakes

These will break your login system — know them in advance

1

Forgetting `session_start()`

Must be line 1 of EVERY file using sessions — before any HTML.

2

echo or HTML before `session_start()`

'Cannot send session cookie — headers already sent'. Move `session_start()` to the very top.

3

Not protecting every restricted page

Each page needs its own guard — session is not automatic.

4

Comparing plain text password in DB

Never store plain text. Use `password_hash()` + `password_verify()`.

5

Forgetting `exit` after `header()` redirect

Without `exit`, the rest of the page continues executing after the redirect.

6

Only `unset()` in logout without `destroy()`

Call both `session_unset()` AND `session_destroy()` for a clean logout.

E

Part E

Cookies

Store small pieces of data in the user's browser — preferences, remembered choices

What is a Cookie?

A small text file stored in the user's browser — NOT on the server

```
<?php
// — Creating a cookie —————
// setcookie(name, value, expiry_timestamp)
setcookie("username", "ahmed", time() + 86400);
// time() = now in seconds
// + 86400 = + 1 day (60 sec * 60 min * 24 hours)
// Cookie lasts 1 day

// Set cookie for 30 days
setcookie("theme", "dark", time() + (30 * 24 * 3600));

// — Reading a cookie —————
if (isset($_COOKIE["username"])) {
    $user = $_COOKIE["username"];
    echo "Welcome back, $user!";
}

// — Deleting a cookie —————
// Set expiry in the past to delete it
setcookie("username", "", time() - 3600);
// value = "" and past expiry = browser deletes it

// IMPORTANT: setcookie() must be called BEFORE
// any HTML output — same rule as header()
?>
```

Cookies — How They Work

- 1 A cookie is stored in the user's BROWSER — they can see and edit it.
- 2 `setcookie()` sends the cookie to the browser — must be called before any HTML output.
- 3 The expiry is a Unix timestamp: `time()` + seconds. `time()+86400` = 1 day from now.
- 4 `$_COOKIE['name']` reads a cookie if it exists — always check with `isset()` first.
- 5 To delete a cookie, set its expiry to the past: `time() - 3600`.
- 6 Cookies survive browser restarts (unlike sessions which end when browser closes).

When to Use Cookies — Practical Examples

Cookies are for preferences and convenience — not for sensitive data

```
<?php
// Use Case 1 — Remember username in login form
// After login, save the username in a cookie
setcookie("remember_email", $email, time() + (7 * 24 * 3600));
// 7-day cookie

// In login.php — pre-fill the email field:
$saved_email = "";
if (isset($_COOKIE["remember_email"])) {
    $saved_email = htmlspecialchars($_COOKIE["remember_email"]);
}
?>
<input type="email" name="email" value="<?= $saved_email ?>">

<?php
// Use Case 2 — Save language preference
setcookie("lang", "fr", time() + (365 * 24 * 3600)); // 1 year

// Use Case 3 — Track last visit
setcookie("last_visit", date("Y-m-d H:i"), time() + (30 * 24 * 3600));

if (isset($_COOKIE["last_visit"])) {
    echo "Your last visit: " . $_COOKIE["last_visit"];
}
?>
```

Cookie Use Cases

- 1 Remember Me checkbox: store email in a cookie so the login form pre-fills next time.
- 2 Language/theme preference: user picks French or dark mode — save as a cookie.
- 3 Last visit: track when user was last on the site — show a welcome-back message.
- 4 NEVER store passwords, IDs, or sensitive data in cookies — users can edit them.
- 5 Use sessions for login state (server-side, secure). Use cookies for preferences (client-side).
- 6 Always htmlspecialchars() when displaying cookie values — user could modify them.

Sessions vs Cookies — Side-by-Side Comparison

Understand the difference so you choose the right one each time

Feature	Session	Cookie
Where is data stored?	On the SERVER (PHP files)	In the USER'S BROWSER
Who can see/edit it?	Only the server	The user can see and edit it
How long does it last?	Until browser closes (default)	As long as you set (days, months)
What do you store?	login state, user ID, role	preferences, remember-me, language
Security level?	High — user can't tamper	Lower — user can modify cookie
Typical PHP variable?	<code>\$_SESSION['key']</code>	<code>\$_COOKIE['key']</code>
When does it start?	<code>session_start()</code> required	<code>setcookie()</code> call required
Use for login?	YES — always use sessions	NO — never store auth in cookies

F

Part F

Security + Debugging

Safe password handling, good habits, and debugging tools

Password Security — hash and verify

NEVER store plain-text passwords — use PHP's built-in hashing functions

```
<?php
// — Registration: hash before saving to DB —————
$plain_password = $_POST["password"];

// password_hash() creates a safe one-way hash
$hashed = password_hash($plain_password, PASSWORD_DEFAULT);
// $hashed looks like: $2y$10$abc123def456...
// It is NOT reversible — that is the point

$sql = "INSERT INTO users (name, email, password)
        VALUES ('$name', '$email', '$hashed')";
mysqli_query($conn, $sql);

// — Login: verify the password —————
// Fetch user from DB first:
$user = mysqli_fetch_assoc(mysqli_query($conn,
    "SELECT * FROM users WHERE email='$email'"));

// Compare: typed password vs stored hash
if ($user && password_verify($plain_password, $user["password"])) {
    // ✅ Password is correct
    $_SESSION["user_id"] = $user["id"];
    header("Location: dashboard.php");
    exit;
} else {
    // ❌ Wrong email or password
    header("Location: login.php?error=1");
    exit;
}
```

Password Hashing — Rules

- 1 `password_hash($pass, PASSWORD_DEFAULT)` — creates a strong bcrypt hash. Use this always.
- 2 The hash is different every time even for the same password — that is by design.
- 3 NEVER use `md5()` or `sha1()` for passwords — they are not safe hashing algorithms.
- 4 `password_verify($plain, $hash)` — returns true if the password matches the hash.
- 5 You never need to 'unhash' — you just verify. Hashing is one-way.
- 6 In your DB: the 'password' column must be `VARCHAR(255)` to hold the full hash string.

Safe Habits for Beginner PHP Projects

You don't need to be a security expert — just follow these basics

Validate all inputs

Check every `$_POST` and `$_GET` field before using it. Use `isset()`, `empty()`, `is_numeric()`.

Sanitize displayed data

Always use `htmlspecialchars()` when echoing any user data to the page.

Cast IDs to int

`(int)$_GET['id']` — turns `'5;DROP TABLE'` into `5`. Safe.

Check `REQUEST_METHOD`

Never run `INSERT/UPDATE/DELETE` unless the request is `POST`.

Never trust user input

Form data, URL params, cookies — all can be faked. Always validate.

Avoid raw SQL concatenation

Building SQL with `string + variable` is risky. Learn prepared statements.

Use prepared statements (intro)

`$stmt = $conn->prepare('SELECT * FROM users WHERE email=?');` — safer SQL.

Errors: hide in production

Show friendly messages to users, log real errors for developer only.

Prepared Statements — Safer SQL (First Look)

The right way to send user data into SQL queries

```
<?php
require_once "db_connect.php";

// — Without prepared statements (risky!) —————
$email = $_POST["email"];
$sql = "SELECT * FROM users WHERE email = '$email'";
// If email = ' OR 1=1 -- ' → returns ALL users!

// — With prepared statements (safe) —————
// Step 1: Prepare the query with ? placeholders
$stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");

// Step 2: Bind the actual value (s = string, i = integer)
$stmt->bind_param("s", $email);

// Step 3: Execute
$stmt->execute();

// Step 4: Get the result
$result = $stmt->get_result();
$user   = $result->fetch_assoc();

// — INSERT with prepared statement —————
$stmt = $conn->prepare(
    "INSERT INTO users (name, email, password) VALUES (?, ?, ?)"
);
$stmt->bind_param("sss", $name, $email, $hashed_password);
$stmt->execute();
?>
```

Prepared Statements — Why and How

- 1 Without them: user input goes directly into SQL — an attacker can inject commands.
- 2 With ? placeholders, PHP separates the query from the data — safe by design.
- 3 `bind_param('s', $var)` binds a string. 'i' = integer, 'd' = decimal, 'b' = binary.
- 4 The order of letters in the type string must match the order of ? in the query.
- 5 You don't need to use `htmlspecialchars()` on input for the query — but still use it on output.
- 6 For now: use prepared statements for SELECT by ID and all INSERT/UPDATE with user data.

Debugging — Find and Fix Problems Fast

Tools every PHP developer uses daily

```
<?php
// — echo — print a value to check it —————
echo $name;
echo "Query: " . $sql;    // see what SQL you're running

// — var_dump() — show value AND type —————
var_dump($name);
// string(5) "Ahmed"

var_dump($_POST);
// array(2) { ["name"]=> string(5) "Ahmed" ["email"]=>... }

// — print_r() — readable array output —————
print_r($row);
// Array ( [id] => 1 [name] => Ahmed [email] => a@b.com )

// — die() — stop and show a message —————
die("Got here — query was: " . $sql);
// Good for checking: did I reach this line?

// — mysqli_error() — see SQL error message —————
$result = mysqli_query($conn, $sql);
if (!$result) {
    die("SQL Error: " . mysqli_error($conn));
}

// — Check connection worked —————
if (!$conn) {
    die(mysqli_connect_error());
}
```

Debugging — Quick Guide

- 1 `echo $var` — check a value exists and what it contains. First tool to reach for.
- 2 `var_dump($var)` — shows type + value. Use on `$_POST` or `$row` to see all keys.
- 3 `print_r($array)` — shows arrays in readable format. Useful for rows from DB.
- 4 `die('message')` — stops execution and prints. Use to find out where code stops working.
- 5 `mysqli_error($conn)` — shows the exact SQL error. Always check if `mysqli_query` returns false.
- 6 Remove all debug echoes and `die()` calls before submitting your project!

G

Part G

Mini Project

Putting it all together — end-to-end workflow for your semester project

Complete Project Workflow — From User to Database

Trace data through your entire application

```
// ===== FULL REQUEST FLOW FOR "ADD STUDENT" =====

// 1. User opens register.php in browser
//    → sees an HTML form

// 2. User fills form and clicks "Save"
//    → browser sends POST to save.php

// 3. save.php runs:
require_once "db_connect.php";           // get $conn
if($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim(htmlspecialchars($_POST["name"]));
    $email = trim($_POST["email"]);

    if(empty($name) || empty($email)){
        header("Location:register.php?err=1"); exit;
    }

    $sql = "INSERT INTO students(name,email)
VALUES('$name','$email')";
    if(mysqli_query($conn,$sql)){
        header("Location:list.php?msg=saved"); exit;
    }
}

// 4. list.php runs SELECT and loops through rows
//    displaying each student in an HTML table
```

```
// 5. Student clicks "Edit" → edit.php?id=3
```

End-to-End Flow Summary

- 1 Every page follows the same pattern:
require_once db → check method → validate
→ query → redirect.
- 2 list.php is the hub: links to edit.php?id=X and
delete.php?id=X for each row.
- 3 edit.php loads data with SELECT WHERE,
shows a pre-filled form, posts to update.php.
- 4 update.php runs UPDATE WHERE id=X and
redirects to list.php.
- 5 delete.php runs DELETE WHERE id=X and
redirects to list.php.
- 6 login/logout wraps everything with session
guards on each protected page.

Project Build Checklist

Use this to track your progress on the semester project

Phase 1 — Setup

- ☐ Create database and table in phpMyAdmin
- ☐ Write db_connect.php with mysqli_connect()
- ☐ Test connection — confirm no errors

Phase 3 — List Records

- ☐ Build list.php — SELECT * → while loop → table
- ☐ Add Edit and Delete links with ?id=X
- ☐ Display htmlspecialchars() on all data

Phase 5 — Delete

- ☐ Build delete.php — DELETE WHERE id
- ☐ Redirect after delete
- ☐ Optional: add JS confirm() dialog

Phase 2 — Add Records

- ☐ Build register.php form (HTML)
- ☐ Build save.php — POST → validate → INSERT
- ☐ Redirect to list.php after success

Phase 4 — Edit/Update

- ☐ Build edit.php — SELECT WHERE id → pre-fill form
- ☐ Build update.php — UPDATE WHERE id
- ☐ Redirect and show confirmation

Phase 6 — Login

- ☐ Build login.php form + check_login.php
- ☐ Store user in \$_SESSION after verify
- ☐ Add guard to every protected page



Part H

Recap + Next Steps

What we learned today and what you need to do before next class

What You Learned Today — Full Recap

You now have everything you need to build your semester project

Recap

A

Forms, GET/POST, validation — the foundation of every PHP page.

File Structure

C

db_connect.php, include/require_once, how project files connect.

Cookies

E

setcookie(), \$_COOKIE, cookie vs session — when to use each.

MySQL CRUD

B

INSERT, SELECT, UPDATE, DELETE with full PHP code + common mistakes.

Sessions + Login

D

session_start(), \$_SESSION, login guard, logout with session_destroy().

Security + Debug

F

password_hash/verify, safe habits, htmlspecialchars, var_dump, die().

Mini Project

G

End-to-end workflow: form → PHP → DB → HTML. Checklist for your project.

You now have all the tools.

Connect forms. Save to database. Protect with sessions. Build your project.

`db_connect.php`

`INSERT`

`SELECT`

`UPDATE`

`DELETE`

`session_start()`

`password_hash()`

`htmlspecialchars()`

`header() + exit`

`require_once`

Good luck with your project!